Deployability of Conventional AI and Quantum Machine Learning Detectors for Authentication-Passing Insider Threats in a 5G System*

Hoseok Kwon[1], Bonam Kim[1], and Ilsun You[1,2]

[1] Kookmin University, Seoul, Republic of Korea
{hoseok1997, kimbona, isyou}@kookmin.ac.kr
[2] KMU Global Research Center for ICT Convergence Security, Kookmin University, Seoul, South Korea

Abstract

5G now carries authentication-passing insider threats that bypass perimeter trust: rogue base stations cloning legitimate certificates, clone user equipment sharing leaked credentials, and co-located relay agents forwarding NAS bytes without any stolen credentials at all. We reproduce four such authentication-passing attack scenarios (Benign, Eavesdrop, Impersonation, Relay) on a 5G testbed integrated with an IMS subsystem, and quantify the deployability of four detectors (a cross-plane rule classifier, XGBoost, a Variational Quantum Classifier, and a Hybrid CN-Q-CN model) along three axes. (T) We decompose pipeline latency into capture, extract, infer, and alert stages, and report slack $= t_{\text{attack_goal}} - t_{\text{detect}}$ per (model, scenario). (P) We contrast passive-tap and inline Network Function placement of the detector at AMF and S-CSCF. (Q) We measure per-inference simulator latency for both quantum models and contrast it with the literature-reported Quantum Processing Unit runtime envelope. Three numbers carry the paper: feature extraction consumes 18–23 ms across all models, an order of magnitude above any model's t_{infer}; inline placement of a Hybrid detector adds $\Delta_{p99} = 109$ ms to the NGAP REGISTER round-trip; and the literature QPU envelope is more than 350× the simulator $p99$ for the deepest model we evaluate. From these measurements we derive operational implications for 5G Zero Trust adoption, the near-term role of Quantum Machine Learning, and the evolution axes of AI-Native 6G.

1 Introduction

5G now carries insider threats that pass perimeter authentication and then misbehave inside the trust boundary. Three operator-relevant mechanisms illustrate the problem: a rogue base station with a cloned cell certificate intercepts the plaintext Non-Access Stratum (NAS) signalling that 5G exposes once Authentication and Key Agreement (AKA) completes; a clone user equipment (UE) reusing leaked credentials registers under the victim's IMS Public User Identity (IMPU) and emits fraudulent Session Initiation Protocol (SIP) traffic; and a pair of relay agents co-located with the legitimate radio and core hosts relays NAS bytes end-to-end without any stolen credential at all. Related mechanisms have already produced direct financial losses through SMS one-time-password interception and SIM-swap abuse of mobile-banking sessions [29, 13].

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 48, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

3GPP has dedicated a study item to false base station countermeasures [2], and the natural response is Zero Trust: continuous, per-transaction verification grounded in 3GPP and NIST guidance [3, 24]. The implicit-trust regime of 5G, in which the network and IMS layers assume integrity for every entity that passes authentication, is no longer sufficient.

Two gaps prevent today's detectors from operating under that regime. First, recent peer-reviewed 5G insider detectors stay within one plane: a control-only telemetry service that catches seven Layer-3 (NAS, RRC, and related signalling) attacks [28], a provenance-graph detector on 5G core logs [21], and an O-RAN cross-layer dApp that fuses the physical (PHY), medium access control (MAC), and Radio Resource Control (RRC) layers but stops at the radio boundary [25]. None of these reports t_{detect} against a scenario-specific $t_{\text{attack_goal}}$, decomposes pipeline latency into capture, extract, infer, and alert stages, or quantifies how an inline Network Function (NF) placement changes NGAP (NG Application Protocol) and IMS round-trip latency.

Second, work on Quantum Machine Learning (QML) for network security has so far been demonstrated on generic intrusion-detection datasets [4, 8, 9]; the deployability of QML inside a 5G core that must respect a sub-second attack-goal envelope has not been measured.

We bridge these gaps by reproducing four authentication-passing attack scenarios on a 5G/IMS testbed and quantifying detector *deployability* (the practical feasibility of running the detector inside a 5G core under its latency and placement constraints) along three axes. Our contributions are measurable:

- **Four-scenario reproduction with explicit attack-success times.** We define $t_{\text{attack_goal}}$, the moment the attack succeeds, per scenario: first visible plaintext NAS (scenario B), first fraudulent SIP under victim IMPU (scenario C), first end-to-end NAS byte across the relay (scenario D). Every run records that moment so that the detector pipeline latency can be compared with the scenario budget.
- **Four-stage pipeline decomposition** into t_{capture}, t_{extract}, t_{infer}, and t_{alert}, measured per (model, scenario) for four detectors: a cross-plane rule, XGBoost (gradient-boosted trees), a Variational Quantum Classifier (VQC), and a Hybrid CN-Q-CN model (classical $\rightarrow$ quantum $\rightarrow$ classical layers).
- **T/P/Q feasibility numbers.** We report *slack* $= t_{\text{attack_goal}} - t_{\text{detect}}$ per scenario (T); passive-tap versus inline-NF NGAP and IMS round-trip overhead (P); and per-inference simulator latency for the quantum models, contrasted with the literature-reported Quantum Processing Unit (QPU) runtime envelope (Q).
- **Operational implications** derived from those numbers for the deployment of 5G Zero Trust, the near-term role of QML, and the evolution axes of AI-Native 6G.

Authentication-passing insider threats in 5G and IMS cannot be judged by accuracy alone. Their detectability depends on where the detector sits in the pipeline, what placement it tolerates, and what hardware it can run on. We reproduce four such threats on a 5G/IMS testbed and measure the deployability of conventional and quantum-machine-learning detectors so that the operational envelope of 5G Zero Trust, the near-term role of QML, and the evolution axes toward AI-Native 6G follow from measurement rather than vision.

2 Background and Related Work

We organise prior work into four areas and, for each, contrast what has been done with what we measure.

2.1 5G Insider Threat Detection

Recent peer-reviewed 5G insider-detection systems operate within a single plane. 5G-Spector deploys an Open Radio Access Network (O-RAN) compliant Layer-3 telemetry service and reports detection accuracy plus CPU and memory overhead on seven control-plane attacks [28]. PROV5GC builds provenance graphs over 5G core logs and detects three attack classes targeting confidentiality, integrity, and availability with low logging overhead [21]. Det-RAN moves into the radio side by fusing PHY, MAC, and RRC features in a near-real-time distributed application (dApp) that achieves 2 ms detection latency at the radio-side Distributed Unit (DU) and 85% accuracy on unseen conditions [25].

These works do not target an insider that has already passed authentication. They also do not report t_{detect} against a scenario-specific $t_{\text{attack_goal}}$ or decompose the four-stage pipeline (capture, extract, infer, alert) across scenarios that span multiple signalling planes (control, signalling, IMS, and IPsec backhaul). This paper measures both.

2.2 Zero Trust Adoption in 5G

Zero Trust principles for mobile networks are described at the architectural level. 3GPP SA3 surveys solution candidates in TR 33.894 and TR 33.794 [1, 3], NIST codifies the per-request trust model in SP 800-207 [24], and a position paper for 5G/6G defines the Monitor–Evaluate–Decide loop inside the O-RAN architecture [22].

Where measurements exist, they cover narrow slices. Vomvas et al. quantify Trusted Execution Environment (TEE) attestation overhead on an Open5GS plus UERANSIM testbed and report +7 ms for UE registration and 2–8% CPU cost [27]. Michaelides et al. measure IPsec and WireGuard latency on the N3 interface and the Service-Based Interface (SBI) of Open5GS [17].

No prior work places an insider detector in line with NGAP or the Cx interface (the Diameter link between IMS CSCF and the Home Subscriber Server) and reports the change in round-trip latency. The present paper measures this overhead for four detector classes.

2.3 Quantum Machine Learning for Network Security

QML detectors are demonstrated on generic intrusion-detection datasets. QML-IDS combines quantum and classical layers and is evaluated on public Network Intrusion Detection System (NIDS) datasets [4]. A recent hybrid pipeline compresses features with a small classical encoder and routes them to 2–4-qubit quantum heads deployed on an IBM Quantum device, evaluated on the NSL-KDD and Ling-Spam datasets [8]. Quantum kernels combined with Support Vector Machines (SVMs) reach 85% hardware accuracy on four attack types of a modern NIDS dataset [9].

These detectors target external traffic anomalies rather than 5G insiders that pass authentication and then act on the NGAP, NAS, or IMS path. Per-inference QPU latency decomposition (queue waiting, transpile, execute, and result fetch) is also not reported alongside the operational $t_{\text{attack_goal}}$ constraints of 5G. This paper measures simulator t_{infer} for VQC and Hybrid CN-Q-CN models and contrasts it with the literature QPU runtime envelope.

2.4 5G Open-Source Testbeds

Open-source 5G stacks make experimentation reproducible. OpenAirInterface provides a flexible end-to-end research platform [19], VET5G constructs a virtual end-to-end testbed targeted

at security education and capture-the-flag exercises [30], and a recent comparison evaluates Open5GS, free5GC, and OpenAirInterface on a common benchmark [5].

These testbeds report reproducibility, throughput, and call-rate metrics. They do not report per-stage capture and extract latency for cross-plane detectors, or passive-versus-inline placement overhead at the Access and Mobility Management Function (AMF) or the IMS Serving Call Session Control Function (S-CSCF). This paper does so on a testbed built from this ecosystem (a 5G core integrated with an IMS subsystem), referred to simply as "the testbed" from now on.

2.5 Summary

Prior 5G insider detection stays single-plane, prior 5G Zero Trust work measures only isolated security mechanisms, prior QML in network security targets generic NIDS datasets, and prior 5G testbeds focus on reproducibility. This paper bridges these four gaps by measuring t_{detect} against $t_{\text{attack_goal}}$, decomposing pipeline latency, quantifying inline NF overhead, and contrasting simulator and literature-QPU execution costs per inference.

3 Threat Model and Testbed

3.1 Attack Scenarios and Success-Time Definitions

We reproduce four attack scenarios distinguished by what the attacker has stolen (cloned cell certificate, leaked UE credentials, or nothing). All four share one constraint: the attacker has already passed primary authentication before any detector observes its first packet, so detection cannot rely on registration failure as a signal. The scenarios are deployment tests. Each one isolates a post-authentication condition that can arise after certificate compromise, credential leakage, or host co-location in an operator, laboratory, or supply-chain environment. This keeps the measurement focused on what happens after authentication has already succeeded.

For every run, we fix a common time origin and two events on the same timeline:

- **Run start ($t = 0$).** The moment stage 1 (Radio Resource Control, RRC) packet capture begins. We use this as the time origin for every measurement in the paper.
- **Attack-success time ($t_{\text{attack_goal}}$).** The moment at which the attacker has achieved the goal of the scenario. What counts as "success" is fixed per scenario and is recorded deterministically as a marker in each run's log file.

The detector pipeline has a processing latency t_{detect} (defined in §4.3). We compare that latency with the scenario time budget using *slack*, slack $= t_{\text{attack_goal}} - t_{\text{detect}}$. Slack is positive when the detector pipeline fits inside the attack-success budget and negative when it exceeds that budget.

Scenario A (Benign) carries baseline 5G and IMS signalling on the testbed: one next-generation NodeB (gNB), two UE instances, and standard NGAP, NAS, IPsec, and IMS REGISTER flows. No attacker capability is present, so no $t_{\text{attack_goal}}$ is defined.

Scenario B (Eavesdrop) models a rogue gNB equipped with a cloned cell certificate and key. The rogue node joins the IPsec backhaul at the strongSwan Security Gateway (SeGW) with the same certificate serial as the legitimate gNB, which manifests as two concurrent IPsec Security Associations (SAs) between SeGW and the gNB cluster. This capability represents certificate life-cycle compromise, mis-issuance, or key theft from a trusted gNB host. The attack succeeds as soon as the rogue gNB sees the first plaintext NAS packet at a local packet-capture process attached to it (a process running on the same host as the rogue gNB container, used here

Table 1: Attack scenarios on the common timeline.

Sc.	Attacker has	Attack-success event ($t_{\text{attack_goal}}$)	Stage	Offset	Approx. time from $t = 0$
A	nothing	(no attack)	—	—	—
B	cloned cell cert.	first plaintext NAS visible at the rogue gNB's local capture process	stage 2 (NAS register + AKA)	950 ms	1.32 s
C	cloned cert. & UE creds.	first fraudulent SIP under victim's IMPU	stage 7 (SIP service)	120 ms	3.97 s
D	nothing (byte relay only)	first NAS byte relayed end-to-end across both relay agents	stage 2 (NAS register + AKA)	650 ms	1.14 s

to dump radio-side traffic for the attacker). The Packet Data Convergence Protocol (PDCP) terminates radio-link encryption at the gNB, so once the rogue gNB joins the backhaul the upper-layer NAS bytes appear in cleartext at that local capture process. The attack-success time falls inside the NAS-register-with-AKA stage (stage 2).

Scenario C (Impersonation) adds a clone UE with the same International Mobile Subscriber Identity (IMSI), AKA key material (Ki), and IMS credentials as a victim user. The clone attaches concurrently with the victim through the rogue gNB. The AMF records repeated Subscription Permanent Identifier (SUPI) context churn, the S-CSCF records concurrent IMPU binding, and the clone sends a fraudulent SIP MESSAGE under the victim's IMPU. This models the post-leakage state of SIM-cloning, provisioning-database exposure, or test-profile reuse. The attack-success time is the moment that first fraudulent SIP packet appears, which falls inside the SIP service stage (stage 7).

Scenario D (Relay) carries no stolen credential. Two relay agents – one co-located with the legitimate gNB host, the other co-located with the AMF host – forward NAS bytes between the gNB and the AMF through a Unix-domain-socket relay channel (a local inter-process socket used as the relay path between the two agents). This capability represents host-level co-location or container escape on both ends of the path. The relay is byte-transparent from the operator's perspective but inflates NAS inter-arrival time variance. The attack-success time is the moment the first NAS byte is relayed end-to-end across both agents, located inside stage 2.

All four scenarios run in RFsim (radio-frequency simulation; no over-the-air radio emission is performed), producing byte-transparent traces on Docker bridges. The measurements focus on the core/IMS timing path: packet form, plane fan-out, feature extraction, and detector placement. Live-radio effects such as fading, scheduler load, handover, mobility, and unrelated subscriber traffic are part of operator extensions.

Table 1 summarises the four scenarios on a single timeline. The run starts at the beginning of stage 1 capture ($t = 0$). The "Stage" column gives the capture stage that contains the attack-success time; "Offset" gives milliseconds after that stage starts; "Approx. absolute time" converts this to an approximate distance from $t = 0$, using a reference run and the deterministic inter-stage gaps recorded in the run marker log. Other runs carry the small jitter described in §4.1.

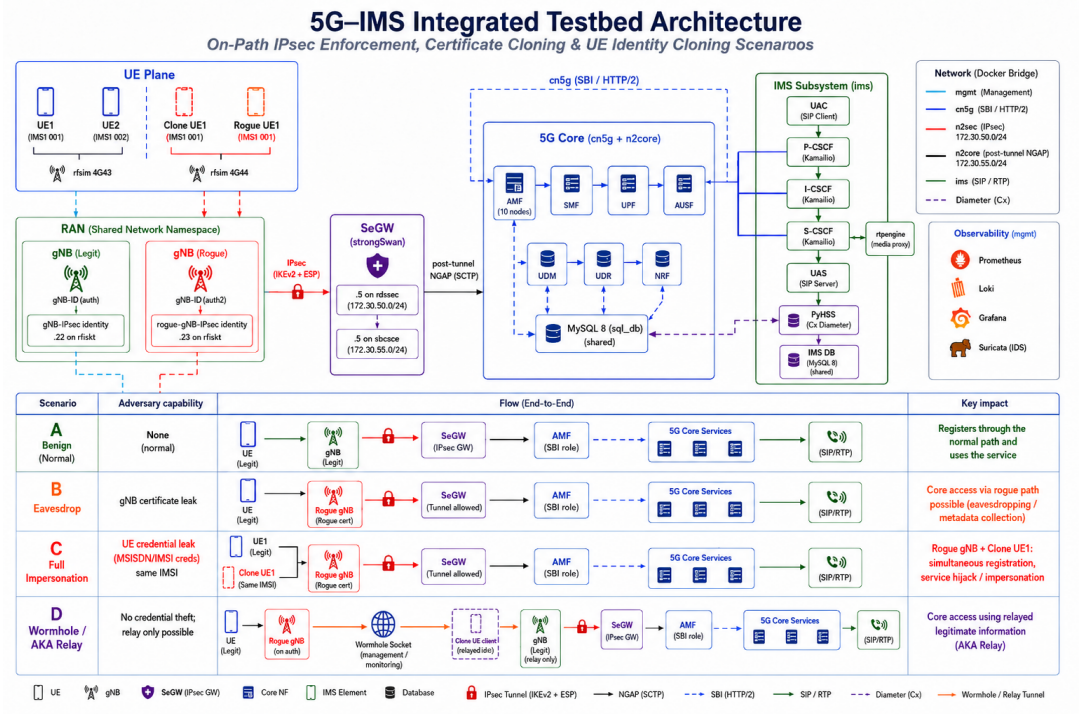


Figure 1: Testbed architecture and attack paths.

3.2 Testbed

Before describing the stack, we fix one term we use throughout the paper. A *signalling plane* (or just *plane*) denotes a logically separated path along which the testbed carries a class of messages. The four planes relevant to our detection are NGAP (control-plane signalling between the gNB and the AMF), NAS (signalling between the UE and the AMF, transparent to the radio access), Cx Diameter (control-plane signalling between the IMS CSCF cluster and the Home Subscriber Server), and the IPsec backhaul (the encrypted transport between the gNB cluster and the SeGW). A *cross-plane signal* is an observable signature derived from more than one of these planes in the same run.

Our reference testbed is the published workshop testbed. Its component stack is detailed in [16, 12, 26] and illustrated in Figure 1: an Open5GS-aligned OpenAirInterface 5G Core, the Kamailio P/I/S-CSCF IMS cluster, and the strongSwan SeGW. The stack is hosted in five Docker bridges: a management bridge; a 5G control-plane bridge `cn5g` for the OpenAirInterface 5G Core network functions (NRF, UDR, UDM, AUSF, AMF, SMF, and UPF); the `n2sec` bridge that carries IPsec ESP between the SeGW and the gNB-side processes; the `n2core` bridge that carries SCTP/NGAP between the SeGW and the AMF; and the `ims` bridge that hosts the Kamailio Proxy, Interrogating, and Serving CSCF cluster, PyHSS over Cx Diameter, and rtpengine for media. The 5G Core and the IMS share a single MySQL subscriber database. All NGAP signalling is forced through the strongSwan SeGW on `n2sec`. Figure 1 shows the bridge layout and the scenario-specific attack paths.

For each scenario we capture eight stages on the appropriate bridge: RRC initiation on `cn5g`, NAS register with AKA on `n2core`, NAS Security Mode Command on `n2core`, PDU session

establishment on `n2core`, IPsec backhaul on `n2sec`, IMS REGISTER on `ims`, SIP MESSAGE and INVITE on `ims`, and RTP media on `ims`. Stage boundaries are seeded deterministically per run. The eight cross-plane signals ("fingerprints" in the workshop reference) reported by the workshop testbed are derived from per-stage packet observations: two timing signals (REG-to-401 latency and IMS REGISTER inter-arrival time) and six state signals (SeGW duplicate SA count, NGAP source IP diversity, distinct gNB-side processes observed on `n2sec`, AMF SUPI replications, S-CSCF concurrent IMPU binding, and synthetic NAS injection events).

The plane fan-out of the testbed (i.e., how a single user action splits across multiple planes), the NGAP and IMS packet ordering, and the inter-arrival distributions of a deployment are traffic-form properties. The detection pipeline reads counts, inter-arrival times, source diversities, and protocol fractions rather than NAS or SIP payload bytes. We therefore generate scenario-derived pcaps whose traffic form (per-stage packet count, plane fan-out, protocol distribution, and inter-arrival shape) matches the scenario specifications, while the payload bytes are randomised. This keeps the experiment aligned with the deployability question measured in this paper.

4 Methodology

4.1 Reproduction Protocol

We generate ten runs per scenario, for a total of forty runs and three hundred and twenty per-stage pcaps. Each run is deterministic in its seed and carries a marker log that records the run identifier, the eight cross-plane signal targets, and the attack-success time $t_{\text{attack_goal}}$ in seconds relative to the run start ($t = 0$, the start of stage 1 capture, as defined in §3.1). Stage boundaries follow the workshop reference described in §3.2; the per-stage baseline packet counts and the per-scenario multipliers that govern attacker-induced traffic-form deviations are encoded in the scenario generator. Inter-stage gaps carry 100–300 ms of pseudo-random jitter (seeded by the run index) so that no two runs have identical absolute times. We use these ten runs per scenario to report central values, observed distributions, and $p99$ timing boundaries. Larger repetition counts add confidence intervals around the same T/P/Q measurements.

4.2 Detection Models

We evaluate four detectors, summarised in Table 2. They are chosen so that classical and quantum, shallow and deeper, and stateless and stateful designs are all represented, and so that per-inference cost differs by orders of magnitude across the four. Principal Component Analysis (PCA) compresses the 200-dimensional packet-form input into the smaller dimension that each quantum model takes as input. Q1 and Q3 are simulated on the PennyLane `default.qubit` backend with the autograd interface [6]; see §4.5 for QPU hardware scope. A Variational Quantum Classifier (VQC) maps input scalars to qubit rotation angles and trains circuit parameters for classification. "AngleEmbedding" is the PennyLane primitive that performs this mapping, and "BasicEntanglerLayers" denotes the nearest-neighbour entangling block used in both quantum models. "CN-Q-CN" denotes the classical-network → quantum → classical-network composition: a classical projection feeds a small quantum circuit, and the four Pauli-Z measurement outputs feed a learned classical head.

The cross-plane rule C0 checks state-based signals first, then the jitter-based signal: it returns scenario C when any AMF SUPI replication is observed or two or more concurrent IMPU bindings are seen, scenario D when the synthetic-injection signal reaches two or more,

Table 2: Detection models and key parameters.

ID	Model	Input	Key parameters
C0	Cross-plane rule	8 cross-plane signals	3 priority rules (state signals first, then jitter signal)
C1	XGBoost [7]	200-dim flat	80 trees, depth 4, learning rate 0.2
Q1	VQC	PCA 200→4, 4 qubits	BasicEntanglerLayers, L=2, 80 epochs
Q3	Hybrid CN-Q-CN	PCA 200→16, linear 16→4, 4 qubits, linear 4→4	BasicEntanglerLayers, L=6, 50 epochs [8]

scenario B when the SeGW SA duplicate count or NGAP source-IP diversity reaches two, and scenario A otherwise. C1 trains a gradient-boosted tree on the flat 200-dimensional feature. Q1 reduces the 200-dimensional feature to four real values via PCA, angle-encodes them onto four qubits, and applies two layers of basic entangler. Q3 inserts a learned linear projection from the sixteen leading principal components to the four-qubit angle space, runs a six-layer basic entangler on four qubits, and feeds the four Pauli-Z expectations into a learned classical head.

All four detectors are evaluated with 5-fold cross-validation (CV) at the run level: every fold holds out one fifth of the runs per scenario, so the same run never appears in both training and evaluation. The feature extractor is fit inside each training fold before it is applied to the held-out runs, so PCA statistics and model parameters are not shared from evaluation data into training. The split evaluates generalisation across held-out runs within the four scenarios.

4.3 Pipeline-Latency Decomposition

The detector pipeline latency is decomposed into four serial stages:

$$t_{\text{detect}} = t_{\text{capture}} + t_{\text{extract}} + t_{\text{infer}} + t_{\text{alert}}.$$

t_{capture} is the wall-clock time to read all eight per-stage pcaps for one run. t_{extract} is the wall-clock time to compute the 200-dimensional packet-form feature vector and the eight cross-plane signals from the same pcaps. t_{infer} is the per-sample model inference time measured with `time.perf_counter` immediately around the single-sample predict call. t_{alert} is a deterministic 0.5 ms write-cost included as a reference floor for any logging stage that follows the model output. The slack between the scenario time budget and the detector pipeline is

$$\text{slack} = t_{\text{attack_goal}} - t_{\text{detect}},$$

reported in milliseconds. Positive slack means the complete detector pipeline fits inside the attack-success budget; negative slack means the pipeline exceeds it.

This definition measures complete-window cross-plane detection. The feature vector is computed after the eight per-stage pcaps are available, so the slack values answer whether the processing cost of the full cross-plane pipeline fits inside the scenario budget. A streaming implementation can reuse the same timeline and report an additional earliest-decision time from incremental windows inside stage 2 or stage 7.

4.4 Feasibility Three Axes

T-feasibility reports the four-stage decomposition stacked per (model, scenario), and the slack distribution per (model, scenario).

P-feasibility contrasts two detector placements. In the *off-path placement*, the detector reads a mirrored copy of the Docker bridge traffic and never sits on the data path, so it cannot block packets; it can only raise alerts after the fact. In the *on-path placement*, every NGAP REGISTER packet at the AMF and every Cx packet at the S-CSCF must pass through the detector before being forwarded to its destination, so each packet incurs the per-packet t_{infer} cost. We report the change in the NGAP REGISTER-to-200 OK and IMS REGISTER-to-200 OK round-trip latency.

Q-feasibility reports per-inference simulator latency on PennyLane `default.qubit` for both quantum models. We contrast it with the literature-reported QPU runtime envelope (the range of expected values for one inference on a physical QPU), broken into *queue* (waiting time before the job runs on the physical device), *transpile* (compiling the abstract circuit to the hardware-native gate set), *execute* (running the compiled circuit on the QPU), and *fetch* (returning measurement results to the client), as cited in §5.3 [9, 23, 18].

4.5 Measurement Boundary

This paper measures controlled deployability. The pcaps preserve packet count, plane fan-out, protocol order, and inter-arrival distribution from the scenario specifications, while payload bytes are randomised. The feature set uses traffic-form statistics, so these pcaps measure the feature-extraction and placement costs that dominate the T/P axes. For the Q axis, the paper measures simulator latency on PennyLane `default.qubit` and compares it with literature-reported physical-QPU runtime components. The same setup can be extended with live PCAP comparisons, larger repetition counts, unseen-scenario tests, federated multi-operator topologies, and RFsim-to-RF translation.

5 Feasibility Analysis

5.1 T-Feasibility: Pipeline-Latency Decomposition and Slack

Figure 2 (left) shows the mean four-stage latency per (model, scenario). Two observations follow directly from the numbers. First, $t_{\text{capture}} + t_{\text{extract}}$ ranges between 18 ms and 23 ms across all four models and all four scenarios, while t_{infer} ranges from 0.8 μs (C0 rule) to 3.5 ms (Q1 VQC) to 7.0 ms (Q3 Hybrid). Feature extraction is the bottleneck, and it is independent of model class. Second, the maximum t_{detect} across all (model, scenario) cells is 60.9 ms ($p99$, Q3 Hybrid on scenario B), where the dominant contribution is the $p99$ capture latency at scenario B, not the inference path.

The slack distribution in Figure 2 (right) is positive across all (model, scenario) pairs for which $t_{\text{attack_goal}}$ is defined. Mean slack ranges from 1105 ms (Q3 Hybrid on scenario D) to 4075 ms (C0 rule on scenario C). The difference in mean slack between C0 and Q3 within a fixed scenario is approximately 8 ms; this matches the mean t_{infer} difference between the two models and confirms that the contraction of slack as the model deepens is fully attributable to inference cost, not to the rest of the pipeline. Across the ten runs per scenario, the observed $p05$–$p95$ slack intervals remain positive: 1264–1468 ms for scenario B, 3773–4327 ms for scenario C, and 1048–1166 ms for scenario D across the four models. These slack values are complete-window latencies: they establish the cost of running the full cross-plane detector over all eight

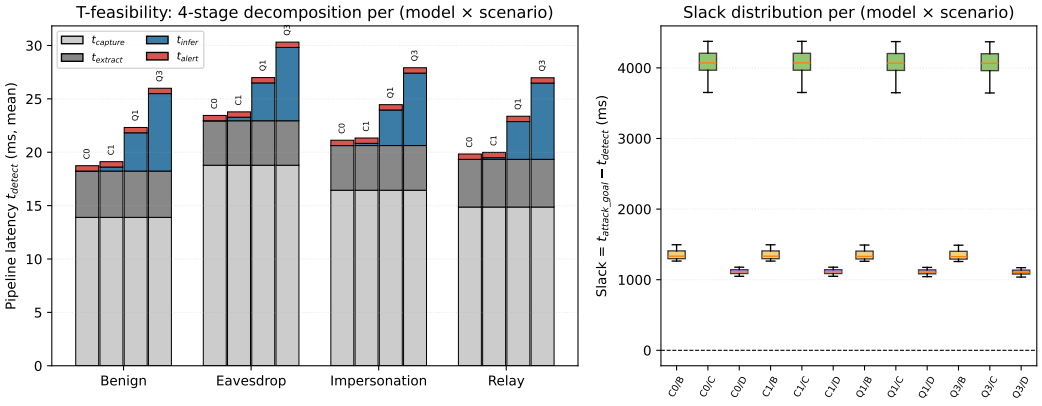


Figure 2: T-feasibility results.

Table 3: Detector accuracy and recall.

Model	Macro F1	Acc.	A	B	C	D	Mean t_{infer}
C0 Rule	0.95	0.95	1.00	1.00	1.00	0.80	0.86 μs
C1 XGBoost	0.97	0.97	1.00	1.00	1.00	0.90	0.27 ms
Q1 VQC	0.41	0.48	0.60	0.10	1.00	0.20	3.50 ms
Q3 Hybrid	1.00	1.00	1.00	1.00	1.00	1.00	7.01 ms

stages. They are the baseline against which an incremental streaming detector can later report earliest-decision time.

The accuracy of the four detectors, computed across the same five-fold run-level split, is summarised in Table 3. The macro-F1 score is the per-class F1 averaged over the four scenarios; it weights each scenario equally regardless of support. C0 rule and C1 XGBoost recover scenarios B and C through their state-based signals; both partially miss scenario D, whose jitter-based signal is harder to capture with a static rule. Q1 VQC at L=2 shows limited capacity for the 4-class problem under the PCA(200,4) input budget (a four-dimensional input compressed from 200 raw stats); this reflects the operational constraint of the shallow circuit depth (see §6.2). Q3 Hybrid CN-Q-CN, with deeper quantum layers and a learned classical head, reaches perfect macro-F1 within the four-scenario corpus. The deployability result is where this accuracy lands inside the T/P/Q timing envelope: higher model capacity improves classification here, but it also moves the detector away from on-path placement.

5.2 P-Feasibility: Off-Path versus On-Path Placement

We model two detector placements. In the *off-path* placement, the detector reads a mirrored copy of a Docker bridge and does not sit on the data path; the NGAP REGISTER and IMS REGISTER round-trip latencies are 5.2/12.4 ms and 9.4/23.1 ms at $p50/p99$ respectively, and the detector cannot block packets, only raise alerts after the fact. In the *on-path* placement, every packet of the relevant handshake is forwarded only after the detector returns a decision: a ten-packet NGAP REGISTER handshake therefore acquires ten t_{infer} samples of overhead, and an eight-packet IMS REGISTER acquires eight.

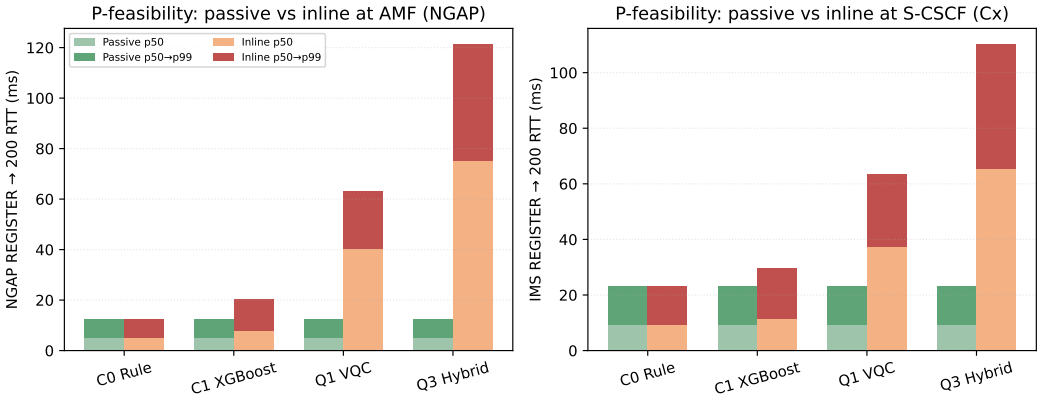


Figure 3: P-feasibility results.

Figure 3 shows the resulting NGAP and Cx round-trip latencies. Bars show the $p50$ latency and the $p50$-to-$p99$ extent. The cross-plane rule and XGBoost remain within an order of magnitude of the baseline: C0 adds +0.04 ms at NGAP $p99$, C1 adds +8.13 ms. Q1 VQC adds +50.5 ms at NGAP $p99$, multiplying baseline by approximately five. Q3 Hybrid adds +109.1 ms, multiplying baseline by approximately ten. The Cx interface shows the same ordering, with smaller absolute deltas because the handshake is shorter.

The split is operational. C0 and C1 are placeable on the data path of the AMF or the S-CSCF; Q1 and Q3 must observe off-path. This is the first quantitative placement result that the present paper establishes. The P-axis stops at detector placement cost; enforcement actuators such as session revocation, slice isolation, and policy update consume the alert downstream.

5.3 Q-Feasibility: Simulator and Literature-QPU Envelope

Figure 4 (left) shows the cumulative distribution of per-inference simulator latency for Q1 and Q3. Figure 4 (right) compares simulator $p99$ with the literature QPU runtime envelope on a logarithmic scale. The simulator $p99$ values are 5.0 ms (Q1) and 10.9 ms (Q3). The four-qubit circuits run on the PennyLane `default.qubit` simulator.

For comparison, prior work that runs the same class of two-to-four-qubit quantum kernels on physical QPUs reports a runtime envelope (the range of expected values for one inference) in which the queue alone consumes between two and thirty seconds, transpile consumes approximately one and a third seconds, execute approximately thirty-five milliseconds, and result fetch approximately one hundred and eighty milliseconds [9, 23, 18]. Figure 4 (right) stacks these literature values on a logarithmic axis next to the measured simulator $p99$. The lower edge of the literature QPU envelope (queue minimum of two seconds plus transpile plus one execute plus fetch) is approximately three and a half seconds; this is more than 350× the Q3 simulator $p99$ of 10.9 ms and more than five hundred times the Q1 $p99$. For the $t_{\text{attack_goal}}$ values of this paper (950 ms after the start of stage 2 for scenario B, 120 ms after the start of stage 7 for scenario C, and 650 ms after the start of stage 2 for scenario D), the QPU runtime envelope sits well above any realistic slack budget. The Q-axis therefore separates two roles: simulator-level QML is useful for model and feature exploration, while physical-QPU execution belongs to offline or auxiliary analysis rather than online per-packet detection. This is the second operational result established by the present paper.

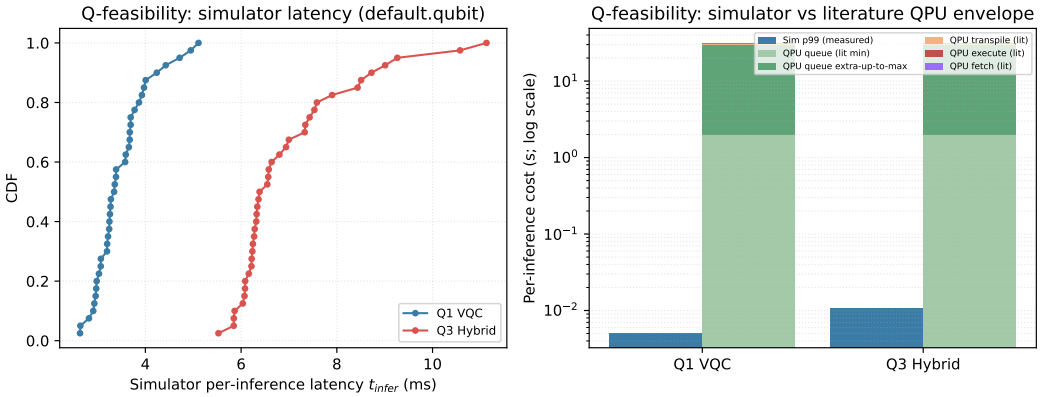


Figure 4: Q-feasibility results.

6 Discussion

The numbers in §5 translate into three implications for the deployment of 5G Zero Trust, the role that Quantum Machine Learning can play in the next few years, and the design axes of AI-Native 6G.

6.1 Operational Constraints on 5G Zero Trust Adoption

Zero Trust as codified in 3GPP and NIST guidance asks that every transaction be verified continuously rather than implicitly trusted after authentication [3, 1, 24]. Our measurements clarify where the verification can sit. The 18–23 ms feature-extraction cost is paid by every cross-plane detector, classical or quantum, so the engineering target most likely to advance deployability is the extraction pipeline itself: parallel tap fan-out, zero-copy reads, and pre-computed protocol counters at AMF and S-CSCF boundaries. On-path placement of an XGBoost-class detector adds 8 ms to NGAP REGISTER $p99$, which fits inside typical 5G latency budgets, but on-path placement of a quantum-augmented detector multiplies that figure by roughly ten and only the off-path placement remains viable. The Zero Trust enforcement of authentication-passing insider threats in 5G therefore divides into two architectural classes: a near-NF, on-path classical class that catches state-based signals (scenarios B and C) within the round-trip envelope, and an off-path, quantum-or-deep class that runs out of band and feeds closed-loop responses (automated detection-to-enforcement feedback loops that revoke a session or isolate a slice after the alert). The measurements in this paper fix the detector side of that loop; session revocation, slice isolation, and policy rollback are the enforcement actuators that consume the alert.

6.2 Near-Term Role of Quantum Machine Learning

Two findings shape the near-term role of QML. First, the shallow VQC at L=2 underperforms on the 4-class problem despite the deeper Hybrid CN-Q-CN reaching perfect F1 on the same dataset. The contrast shows that the small quantum core is sensitive to input compression and model capacity under the feature budget used here. Hybrid pipelines that put classical pre- and post-processing around a small quantum core follow the pattern recently reported on physical hardware [8]. Second, even when the simulator $p99$ is well within the slack budget, the literature-reported QPU queue and transpile costs alone exceed every $t_{\text{attack_goal}}$ in this paper [23]. The

viable role for QML in 5G insider detection over the next few years is therefore *scheduled offline work* (e.g., daily or weekly batch runs that extract quantum-derived features for a downstream classical detector, or pre-train model weights that the classical detector loads), not per-packet online inference. This is a narrower role than the QML-IDS demonstrations report [4], and the boundary is set by the QPU runtime envelope rather than by accuracy.

6.3 Operator Extensions

The measurements isolate the core/IMS detector path. Operator extensions add deployment variables outside that path: live radio variation, mobility, handover, unrelated subscriber traffic, vendor-specific Network Function scheduling, and operator policy diversity. Those variables can move both sides of the slack equation by changing $t_{\text{attack_goal}}$ and t_{detect}. Live PCAP comparisons, larger run counts with confidence intervals, and multi-topology replication extend the present measurement to those operator-facing settings.

6.4 Evolution Axes Toward AI-Native 6G

IMT-2030 sets the platform target for sixth-generation networks at hyper-connected service integration [11], the ten-nation joint statement adds a policy expectation that 6G be secure, open, and resilient by design [10], and the industry security roadmap aligns 6G with built-in trust assumptions [20]. Our measurements identify three axes that have to be addressed before AI-Native 6G can extend Zero Trust enforcement to the radio and edge layers.

First, feature extraction must be amortised closer to the plane: pushing cross-plane counters onto smart Network Interface Cards (smart NICs) or extended Berkeley Packet Filter (eBPF) fast-path hardware is the lever for shrinking the dominant 18–23 ms cost that we measured. Second, on-path decisions must be packaged so that the per-packet t_{infer} is not paid on the data path of every handshake packet. Sampling at the session boundary (running the detector once per registration or call, not once per packet) or returning the decision asynchronously to the data path becomes a deployment requirement, not a future direction.

Third, the foundation of authentication-passing detection is authentication itself. Forward-secret and post-quantum-hybrid extensions to 5G-AKA reduce the insider window that the present detectors must react to [31, 14, 15]. None of these three axes is conjectural; each follows from a number reported in §5.

7 Conclusion

This paper reproduced four authentication-passing attack scenarios on a 5G/IMS testbed built with RFsim and Docker. It decomposed detection latency into capture, extract, infer, and alert stages for four detectors: a cross-plane rule, XGBoost, a Variational Quantum Classifier, and a Hybrid CN-Q-CN model. Three numbers carry the result. Feature extraction consumes 18–23 ms regardless of model class, an order of magnitude above any model's t_{infer} in this setup. On-path placement of a Hybrid quantum detector adds 109 ms to NGAP REGISTER $p99$, multiplying the off-path baseline by approximately ten, so only C0 and C1 remain inline-viable on the AMF and S-CSCF data paths. The lower edge of the literature-reported QPU runtime envelope is more than 350× the Q3 simulator $p99$, placing physical-QPU inference outside the slack budget of every scenario.

Within these numbers, three operational implications follow. First, authentication-passing insider detection in the 5G core is constrained by feature extraction and detector placement,

not by the choice between classical and quantum inference. Second, Zero Trust deployment divides into two architectural classes: a near-NF, on-path classical class that catches state-based signals within the round-trip envelope, and an off-path, deeper or quantum-augmented class that runs out of band and can feed later enforcement actuators. Third, the near-term viable role for Quantum Machine Learning is scheduled offline work, such as extracting quantum-derived features or pre-training a downstream classical detector. It is not per-packet online inference; the boundary is set by the QPU runtime envelope rather than by accuracy.

Future Work

The measurements in this paper lead to three direct extensions.

Model-space expansion. The detector set should be expanded on both sides of the classical–quantum boundary: deeper VQCs, Quantum Kernel SVMs, and Quantum Graph Neural Networks on the quantum side, and BiLSTM, Transformer, and graph-based detectors on the classical side. The aim is to map how the capability boundary observed here moves across model classes and architectures, and where each new model lands inside the T/P/Q envelopes reported in this paper. This extension should include leakage checks, ablations over PCA dimension and qubit count, and unseen-scenario tests that hold out an entire attack family or operator topology.

Hardware validation for QML. The Q-feasibility numbers reported here pair PennyLane `default.qubit` measurements with literature-cited QPU envelopes. Measuring queue, transpile, execute, and fetch directly on physical hardware (e.g., IBM Quantum or IonQ) for the four-qubit Hybrid circuit would either confirm or revise the 350× simulator-to-QPU gap, and would let us check whether the QPU envelope itself shrinks as cloud-QPU schedulers and transpilers evolve. This measurement is the next step for deciding whether QML moves beyond the scheduled-offline role identified above.

Operator extensions. The next measurement step is to move from controlled traces to operator-facing settings. More repetitions across multiple seeds would turn the observed latency and slack distributions reported in §5.1 into confidence intervals. Live PCAPs from the workshop testbed would compare operator traces with the traffic-form dataset described in §4.5. An eBPF or smart-NIC implementation of the cross-plane feature extractor would show how much of the dominant 18–23 ms cost can be removed from the data path. The same follow-up should report earliest-decision latency from incremental feature windows, separately from the complete-batch latency used in this paper.

Acknowledgment

This work was supported by Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIP) (No.2024-00437252, Development of anti-sniffing technology for mobile communication and AirGap environments).

References

[1] 3GPP SA3. Tr 33.894: Study on applicability of the zero trust security principles in mobile networks (release 18). Technical report, 3GPP, 2023. V18.0.0.

[2] 3GPP SA3. TR 33.809: Study on 5G security enhancements against false base stations (FBS). Technical report, 3GPP, 2024. Release 18 study item; framework for measurement-report-based FBS detection.

[3] 3GPP SA3. Tr 33.794: Study on enablers for zero trust security (release 19). Technical report, 3GPP, 2025. V19.1.0.

[4] Diego Abreu, Christian Esteve Rothenberg, and Antonio Abelem. QML-IDS: Quantum machine learning intrusion detection system. In *Proceedings of the 29th IEEE Symposium on Computers and Communications (ISCC)*, 2024. arXiv:2410.16308; classical NIDS datasets only.

[5] Maria Barbosa, Marcelo Silva, Ednelson Cavalcanti, and Kelvin Dias. Open-source 5G core platforms: A low-cost solution and performance evaluation. In *Proceedings of the International Conference on Information Networking (ICOIN)*, 2025. arXiv:2412.21162; Open5GS/free5GC/OAI control-plane latency comparison.

[6] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, M. Sohaib Alam, Shahnawaz Ahmed, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. In *arXiv preprint*, 2018. arXiv:1811.04968.

[7] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD Int'l Conf. on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[8] Zisheng Chen, Zirui Zhu, and Xiangyang Li. Cyber threat detection enabled by quantum computing. *arXiv preprint*, 2025. arXiv:2512.18493; 2-4 qubit hybrid MLP+QSVM/VQC; IBM Quantum deployment with readout mitigation.

[9] Mary L. Cotrupi and Brian R. Callahan. Towards practical quantum kernels for network intrusion detection. In *Proceedings of the AAAI Symposium Series (QIML)*, volume 7, 2025. Vol. 7 no. 1; 85% hardware accuracy on QPU runs; sim vs hardware comparison.

[10] Governments of Australia, Canada, Czech Republic, Finland, France, Japan, Republic of Korea, Sweden, United Kingdom, United States. Joint statement endorsing principles for 6g: Secure, open, and resilient by design. https://www.ntia.gov/other-publication/2024/joint-statement-endorsing-principles-6g-secure-open-and-resilient-design, February 2024. Ten-nation joint statement released at Mobile World Congress, Barcelona.

[11] ITU-R. IMT-2030 Framework for 6G Mobile Systems. https://www.itu.int/en/ITU-R/study-groups/rsg5/rwp5d/imt-2030/, 2023. Accessed: 2025-09-30.

[12] Kamailio Project. Kamailio – open source SIP server. https://www.kamailio.org, 2024. P/I/S-CSCF deployment, v5.8, accessed 2026-05-13.

[13] Myounghoon Kim, Joon Suh, and Hunyeong Kwon. A study of the emerging trends in SIM swapping crime and effective countermeasures. In *2022 IEEE/ACIS 7th International Conference on Big Data, Cloud Computing, and Data Science (BCD)*, pages 240–245, 2022.

[14] Yongho Ko, I Wayan Adi Juliawan Pawana, and Ilsun You. 5g-aka-hpqc: Hybrid post-quantum cryptography protocol for quantum-resilient 5g primary authentication with forward secrecy. *arXiv preprint arXiv:2502.02851*, 2025.

[15] Hoseok Kwon, Yongho Ko, I Wayan Adi Juliawan Pawana, and Ilsun You. Toward quantum-safe 6g mobile networks: A survey on EAP-AKA' family with formal security analysis and empirical evaluation. *IEEE Transactions on Consumer Electronics*, 2025.

[16] Sukchan Lee and Open5GS Contributors. Open5GS: Open source implementation for 5GC and EPC. https://github.com/open5gs/open5gs, 2024. Release-17 alignment, accessed 2026-05-13.

[17] Sotiris Michaelides, Jonathan Mucke, and Martin Henze. Assessing the latency of network layer security in 5G networks. *arXiv preprint*, 2025. arXiv:2505.07328; N3 and SBI IPsec/WireGuard latency on Open5GS testbed.

[18] Paul D. Nation, Abdullah Ash Saki, Sebastian Brandhofer, Luciano Bello, Shelly Garion, Matthew Treinish, and Ali Javadi-Abhari. Benchmarking the performance of quantum computing software. *Nature Computational Science*, 2025. Benchpress framework; arXiv:2409.08844.

[19] Navid Nikaein, Mahesh K. Marina, Saravana Manickam, Alex Dawson, Raymond Knopp, and Christian Bonnet. OpenAirInterface: A flexible platform for 5G research. *ACM SIGCOMM Computer Communication Review*, 44(5):33–38, 2014.

[20] Karl Norrman, Bengt Sahlin, Ben Smeets, Erik Thormarker, and Eva Fogelström. 6g security – drivers and needs. Technical report, Ericsson, May 2024. White Paper GFTL-24:000588.

[21] Harsh Sanjay Pacherkar and Guanhua Yan. PROV5GC: Hardening 5G core network security with attack detection and attribution based on provenance graphs. In *Proceedings of the 17th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec)*, Seoul, Republic of Korea, 2024.

[22] Keyvan Ramezanpour and Jithin Jagannath. Intelligent zero trust architecture for 5G/6G networks: Principles, challenges, and the role of machine learning in the context of O-RAN. *arXiv preprint*, 2022. arXiv:2105.01478; position paper, MED components, no measurements.

[23] Salonik Resch, Anthony Gutierrez, Joon Suk Huh, Srikant Bharadwaj, Yasuko Eckert, Gabriel Loh, Mark Oskin, and Swamit Tannu. Accelerating variational quantum algorithms using circuit concurrency. *arXiv preprint*, 2021. arXiv:2109.01714; VQA QPU sampling overhead.

[24] Scott Rose, Oliver Borchert, Sean Mitchell, and Robert Chandramouli. Zero trust architecture. Technical report, NIST Special Publication 800-207, 2020.

[25] Alessio Scalingi, Salvatore D'Oro, Francesco Restuccia, Tommaso Melodia, and Domenico Giustiniano. Det-RAN: Data-driven cross-layer real-time attack detection in 5G open RANs. In *Proceedings of the IEEE INFOCOM*, pages 41–50, Vancouver, BC, Canada, 2024.

[26] Andreas Steffen and strongSwan Project. strongSwan – IPsec-based VPN solution. https://www.strongswan.org, 2024. IKEv2 + ESP for N2 backhaul, v5.9, accessed 2026-05-13.

[27] Marinos Vomvas, Norbert Ludant, and Guevara Noubir. Establishing trust in the Beyond-5G core network using trusted execution environments. *arXiv preprint*, 2024. arXiv:2405.12177; Open5GS+UERANSIM testbed; Zero Trust Execution model.

[28] Haohuang Wen, Phillip Porras, Vinod Yegneswaran, Ashish Gehani, and Zhiqiang Lin. 5G-Spector: An O-RAN compliant layer-3 cellular attack detection service. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2024. MOBIFLOW + MOBIEXPERT; 7 layer-3 attacks; single-plane (control) scope.

[29] Ying-Tso Wen, Sz-Hsien Wu, Chia-Yi Chen, Yen-Yuan Hsu, Po-Hsueh Wang, Shun-Min Hsu, Chen-Yuan Chang, and Bo-Chen Kung. Real-time rogue base stations detection system in cellular networks. In *Advanced Information Networking and Applications (AINA 2024)*, volume 204 of *Lecture Notes on Data Engineering and Communications Technologies*, pages 465–474. Springer, 2024.

[30] Zhixin Wen, Harsh Sanjay Pacherkar, and Guanhua Yan. VET5G: A virtual end-to-end testbed for 5G network security experimentation. In *Proceedings of the 15th Workshop on Cyber Security Experimentation and Test (CSET)*, pages 19–29, 2022.

[31] Ilsun You, Gunwoo Kim, Seonghan Shin, Hoseok Kwon, Jongkil Kim, and Joonsang Baek. 5g-aka-fs: A 5g authentication and key agreement protocol for forward secrecy. *Sensors*, 24(1):159, 2024.